Informe de Entrega - Evaluación Parcial N°1: Tu primer pipeline de despliegue

Asignatura: Ingeniería DevOps (DOY0101)
Docente: Cristian Andrés Bugueño Pantoja
Fecha de Entrega: 04/09/2026

Integrantes del Equipo y Roles

1. **Kevin Gallardo** - Frontend Developer
2. **Alexsander Rosales** - Backend Developer
3. **Paulo Rivas** - DevOps Engineer

---

1. Enlace al Repositorio

El desarrollo de este encargo se encuentra en el siguiente repositorio de GitHub: **Enlace:** https://github.com/iTzAsaro/prueba-devops1

(Recuerden actualizar el enlace antes de entregar)

---

2. Desarrollo del Encargo

En el repositorio enlazado se ha dado cumplimiento a los siguientes requerimientos de la rúbrica:

Estructura de Ramas y Estrategia de Ramificación

Se implementó formalmente el modelo **GitFlow**. Se emplean dos ramas principales (`main` y `develop`), y ramas auxiliares estrictamente tipificadas (`feature/*` , `hotfix/*` , `release/*`).
Justificación:

1. **Aislamiento por capas:** Permite trabajar en funcionalidades paralelas (Frontend y Backend) sin afectar el código que está listo para entrega ni bloquear el trabajo mutuo.
2. **Ambiente de Integración (`develop`):** Proporciona un entorno controlado donde probar la integración de ambos subsistemas antes de pasar a producción (`main`).
3. **Respuesta Rápida y Segura a Incidentes (`hotfix/*`):** Permite corregir fallos críticos directamente desde `main` y propagar la solución simultáneamente hacia `develop` , garantizando consistencia absoluta en el historial.

Simulación Colaborativa

Se realizaron y fusionaron exitosamente *Pull Requests* (evidenciados en la bitácora del repositorio):

- **PR #1 (Feature):** `feature/backend-api-endpoints` hacia `develop` (Alexsander Rosales).
- **PR #2 (Feature):** `feature/frontend-dashboard` hacia `develop` (Kevin Gallardo).
- **PR #3 (Hotfix):** `hotfix/fix-client-cors-error` hacia `main` y `develop` (Kevin Gallardo / Paulo Rivas).

Automatización CI/CD

Se configuró un flujo de *GitHub Actions* (`ci.yml`) en un entorno de Ubuntu.

- **Eventos:** Se dispara con cada `push` hacia `develop` (para validar integración de forma temprana) y `pull_request` hacia `main` (actuando como compuerta de calidad antes de pasar a producción).

Documentación de Buenas Prácticas

En los archivos `README.md` y `CONTRIBUTING.md` se han detallado las convenciones de *commits* (Conventional Commits), flujos de merge, nomenclatura de ramas y estrategias de revisión.

---

3. Declaración de Uso Ético de Inteligencia Artificial (IA)

En cumplimiento de las directrices académicas institucionales de Duoc UC:

- **Herramientas utilizadas:** Asistente de IA (Gemini / Antigravity CLI).
- **Uso aplicado:** Apoyo en la diagramación de flujos, estructura de plantillas de documentación en Markdown y generación de esqueletos base de código.
- **Validación:** Todos los análisis técnicos, diseño de arquitectura GitFlow, implementación de componentes y reflexiones fueron revisados, validados y adoptados de manera propia por el equipo de estudiantes.

---

4. Reflexiones Individuales (Conclusiones)

Reflexión de Kevin Gallardo (Frontend Developer)

> *Como Frontend Developer, mi principal desafío fue construir una plataforma de preventa inmersiva (SPA vanilla JS con ES Modules) sin frameworks ni bundlers, lo que me obligó a diseñar un router propio, un store con persistencia en LocalStorage y un design system dark HUD desde cero. La integración del flujo transaccional — customizer, checkout multi-paso con validación Luhn y dashboard de gestión— requirió un manejo de estado cuidadoso para no perder la configuración al recargar. La colaboración bajo GitFlow mediante ramas `feature/*` y Conventional Commits me permitió entregar el trabajo de forma incremental y trazable, abriendo Pull Requests hacia `develop` que pasaron las compuertas de CI. Esta experiencia consolidó mi comprensión de cómo la arquitectura modular frontend y la gobernanza de versiones se complementan para entregar valor de forma segura y colaborativa.*

Reflexión de Alexsander Rosales (Backend Developer)

> *En mi rol como Backend Developer, el principal aprendizaje radicó en diseñar un microservicio ligero, modular y resiliente que se integrara de manera transparente con el frontend y con los estándares DevOps del equipo. Implementar la API REST nativa —abarcando endpoints de observabilidad como `/api/health` y `/api/metrics`, junto con la gestión completa del ciclo de vida de preventas (`/api/orders`) y el catálogo de productos— me permitió comprender la importancia crítica de la consistencia en los contratos de datos, el manejo seguro de cabeceras CORS y el logging estructurado de peticiones HTTP. La adopción rigurosa de GitFlow mediante ramas de características (`feature/backend-api-endpoints`) y Conventional Commits facilitó una trazabilidad total del código fuente, mientras que la construcción de una suite exhaustiva de 10 pruebas unitarias e integración garantizó que cada cambio hacia `develop` pasara automáticamente las compuertas de calidad de CI/CD antes de ser integrado. Esta experiencia consolidó mi visión sobre cómo las buenas prácticas de desarrollo ágil y automatización minimizan errores en producción y potencian el trabajo colaborativo en la nube.*

Reflexión de Paulo Rivas (DevOps Engineer)

> *Durante este proyecto, el desafío principal como DevOps Engineer fue orquestar el flujo de colaboración para asegurar que el trabajo en paralelo no afectara la estabilidad del código base. Al optar por GitFlow, logramos aislar las nuevas características en ramas* `feature/*` *e integrarlas en* `develop` *, brindando un entorno seguro para las pruebas. La configuración del pipeline en GitHub Actions fue clave para la Integración Continua (CI), ya que automatizó la ejecución de las suites de pruebas tanto del frontend como del backend ante cada push a* `develop` *y en los pull requests hacia* `main` *. Adicionalmente, el haber dockerizado los microservicios y levantado el entorno con* `docker-compose` *asegura que cualquier integrante o proceso de CI/CD trabaje bajo las mismas condiciones y dependencias, eliminando el clásico problema de "en mi máquina sí funciona". Esta práctica consolida el trabajo en equipo mediante trazabilidad, automatización y portabilidad.*